MDSPAN

Document #:	P0009r18
Date:	2022-07-13
Project:	Programming Language C++ LWG
Reply-to:	Christian Trott <crtrott@sandia.gov> D.S. Hollman <me@dsh.fyi> Damien Lebrun-Grandie <lebrungrandt@ornl.gov> Mark Hoemmen <mark.hoemmen@gmail.com> Daniel Sunderland <dansunderland@gmail.com> H. Carter Edwards <hedwards@nvidia.com> Bryce Adelstein Lelbach <brycelelbach@gmail.com> Mauro Bianco <mbianco@cscs.ch> Ben Sander <ben.sander@amd.com> Athanasios Iliopoulos <> John Michopoulos <> Nevin Liber <nliber@anl.gov>

Contents

Special thanks to Tomasz Kaminski for invaluable help in preparing this paper for wording review.

1 Revision History

1.1 P0009r18: 2022-07 Mailing

1.1.0.1 Changes from R17

- Wording changes based on LWG feedback

1.2 P0009r17: 2022-05 Mailing

1.2.0.1 Changes from R16

- significant wording updates based on LWG feedback
- `submdspan` was moved into its own paper, allowing mdspan to go forward even if there is not enough time to review `submdspan`
- Merge P2553 (SizeType as a template parameter for `extents`).
- Merge P2554 and fix merge conflict with P2553
- (prior to removal) fixed precondition violation in submdspan when dealing with some types size 0 multidimensional index spaces
  - e.g. `submdspan(a,typle{a.extent(0),a.extent(0)})`
  - thanks to sarah el kazdadi (sarah.elkazdadi@gmail.com) for pointing out this UB behavior

1.3 P0009r16: 2022-03 Mailing

1.3.0.1 Changes from R15

- significant wording updates based on LWG feedback

1.4 P0009r15: 2022-02 Mailing

1.4.0.1 Changes from R14

- `mdspan::rank[_dynamic]` returns `size_t`
- fix comparison operator for `layout_stride` to take strides into account.
- fix `layout_stride` mapping `required_span_size`
- Consistently use "<*expr*> is `true`" instead of "<*expr*> is true".

- In the layout mappings' `operator()` *Effects* clauses, use only `index_sequence`and fix syntax error in `stride(P())`.
- Replace various unary folds with binary to handle `extents<>`.
- Consistently use `Extents::rank()` in layout mapping wording.
- Typo in `mdspan::mapping_type`: `template mapping_type` should be `template mapping`.
- `mdspan`'s `array` constructor calls `Extents`'s `array` constructor.
- Clarify the value of `static_stride` for `submdspan`'s result.
- Significant editorial changes based on LWG small-group review

1.5 P0009r14: 2021-11 Mailing

1.5.0.1 LEWG Review 11/01/2021

- ACTION: Maybe a default constructible accessor should imply default constructible spans. Default constructible spans are important in many cases.
  - done
- ACTION: Make extents conditionally explicit when converting from dynamic to static extents.
  - done
- ACTION: Make layout and mdspan converting constructors conditionally explicit based upon the underlying types
  - done
- ACTION: constructing extents from std::array needs a deduction guide?
  - not done: you can't do this since you can't use alias and you can't return a parameter pack
- QUESTION: Extents parameter pack ctor needs to be explicit? Deduction guide needs to be explicit?
  - we made that change
- QUESTION: submdspan constructor can take tuple<size_t, size_t> and get pair<size_t, size_t> for free (could add specific type for submdspan, so long as it can be converted from pair and tuple).
  - we changed submdspan to take things convertible to `tuple<size_t, size_t>`
- ACTION: needs feature test macro
  - done
- QUESTION: Can get rid of mdspan ctor takes pointer + array? Can construct extent from array. Disagreement on authors on this one.
  - not done: would remove ctad from array
- ACTION: Explore modifying the requirements on which extents need to be provided when constructing an mdspan or extents object.
  - done
  - Authors decided to enable construction from both dynamic extents only, and all extents (for both integer packs and arrays)
  - Why from dynamic extents only:
    - no redundant information
    - precondition free constructor
    - enables fully static extents mdspan construction from ptr only
  - Why from all extents:
    - no confusion what extent a given argument is associated with
    - enables easier writing of certain types of generic code e.g.:

```
template<class mds1_t, class mds2_t>
auto alloc_gemm_result(mds1_t mdspan1, mds2_t mdspan2) {
   using return_t = mdspan<double,
     Extents< mds1_t::extents_type::static_extent<0>,
              mds2_t::extents_type::static_extent<1>>;
   double* ptr = new double[mdspan1.extent(0)*mdspan2.extent(1)];
   return return_t(ptr, mdspan1.extent(0), mdspan2.extent(1));
 }
```

1.5.0.2 Changes from R13

- changes to harmonize with `std::span`
  - made `extents` converting constructor conditionally explicit, for cases where dynamic extents are turned into static extents
  - made convertibility of `default_accessor` depend on convertibility of `element_type(*)[]` instead of `pointer` to prevent derived class to base class assignment
  - remove converting assignment operators throughout
- made layout mapping converting constructors conditionally explicit, depending on `extents` being not implicitly convertible
- made `mdspan` converting constructor conditionally explicit, for cases where any of the exposition only members or the template parameters are only explicitly convertible
- Improve submdspan wording
  - the wording defines more clearly how the submdspan is constructed, not just through ensures
- made layout wording style consistent
- don't require default constructibility from accessors and mappings (still require it for pointer though)
- fixed layout_stride conversion construction
- made deduction guide from integers for extents/mdspan explicit
- tweaked constraints on mdspan to not include element type and the full extents
  - left pointer, since mdspan converts those in its converting constructor
  - also left some specific constraints regarding extents to prevent custom layouts from changing rank or assigning different sized static extents
- add feature test macro
- accept `tuple` instead of `pair` for subslice arguments in `submdspan`.
- remove `mdspan::unique_size`
- fix `layout_stride` constructor to be flexible with integral types of strides array
- make `extents` and `mdspan` constructors accept either `rank_dynamic` or `rank` integer arguments (or an `array` of that size)
- remove mdspan trivially default constructible clause: it never is because we value initialize pointer inline
- remove *nonowning* word from mdspan description: there is not really a reason to have it. Would allow `shared_ptr` as `pointer`
- allow conversion for 1D `layout_left` to `layout_right` and vice versa
- allow implicit conversion for rank-0 `layout_left`, `layout_right`, and `layout_stride` to each other

1.6 P0009r13: 2021-10 Mailing

LEWG reviewed P0009r12 together with P2299r3 on 2021-06-08.

LEWG Poll Approve the direction of P2299R3 and merge it into P0009.

SF	F	N	A	SA
12	6	0	1	0

Attendance: 25; Number of authors: 1 [presumably for P2299, as P0009 coauthors were also attending]; Author's Position: SF.

- Incorporated changes proposed by P2299r3
  - Added `dextents` alias
  - Removed `mdspan` alias and renamed `basic_mdspan` to `mdspan` (which is now a class type, not an alias). This undoes a change introduced in P0009r6. P2299r3 explains the rationale. Existing code using the `mdspan` alias will need to change by replacing the list of extents template arguments with a single `extents` type.
  - Added `mdspan` deduction guides
  - As needed for deduction guides, added `layout_type` alias to layout mapping requirements and to `layout_left`, `layout_right`, and `layout_stride`
- Adapted new LWG wording guidelines by replacing "Expects" with "Preconditions"
- Minor formatting corrections
- Added design discussion as requested by LEWG

— Remove unconditional `noexcept` from `mdspan`

— Fix layout `required_span_size` for rank-0 mdspan

— Fix `layout_stride::required_span_size` for mdspans with at least one extent being zero

— Use `operator[]` in `mdspan` for multidimensional array access, and add explanation to Discussion section

— Remove reference to `span` in the `mdspan` wording, since `mdspan` does not necessarily require a backing `span` (because `pointer` need not be `ElementType*`)

— added conversion constructor for strided and unique layouts to `layout_stride`

— added constructor for `mdspan` from `pointer` and `extents`

— added requirement for layout policy mapping to be nothrow move constructible and assignable

— added requirement for accessor policy to be nothrow move constructible and assignable

— added requirement for accessor policy pointer to be nothrow move constructible and assignable

— remove throws nothing clauses from mdspan and submdspan.

1.7 P0009r12: post 2021-05 Mailing

— Fixed definition of `static_extent`

— Added converting constructor for `default_accessor` (when the pointers to elements are convertible) for things like `default_accessor<double>` to `default_accessor<const double>`

— Changed [mdspan.accessor.basic] to [mdspan.accessor.default], to correspond with the name change in P0009r11

— Minor formatting corrections

1.8 P0009r11: 2021-05 Mailing

— Ask LEWG to poll on targeting P0009 for C++23

— Change all the sizes from `ptrdiff_t` to `size_t` and `index_type` to `size_type`, for consistency with `span` and the rest of the standard library`

— Renamed `IndexType` to `SizeType` or `SizeTypes` (depending if it is a single type or a parameter pack)

— Changed comparisons to hidden friends

— Explicitly mention which types are trivially copyable or empty. This is important as a major intended use case for this is heterogeneous computing. A trivially copyable is heavily used as a proxy for types which can be copied between a host (such as a CPU) and a device (such as a GPU) or between two devices by just copying the bytes which make up the object representation of the type. If they are not trivially copyable, heterogeneous computing would not be able to use these types as vocabulary types.

— State the conditions that make `basic_mdspan` trivially default constructible

— In `layout_*` types, made `operator()` and `stride()` constexpr

— In `layout_stride`, made assignment operators and `required_span_size()` constexpr to match the other `layout_*` types

— Renamed `subspan` to `submdspan`, as this only applies to `mdspan`

— Made `submdspan()` constexpr

— Tweak the wording of `is_strided`

— Renamed `all_type` to `full_extent_t` and `all` to `full_extent`

— Renamed `accessor_basic` to `default_accessor`

— Removed accessor policy `decay(p)` member function as it was an artifact from an earlier version of this proposal when `basic_mdspan` had a `span()` member function that returned a `std::span`

— Removed `span()` from [mdspan.basic.members] description as `.span()` was removed from an earlier version of this proposal

1.9 P0009r10: Pre 2020-02-Prague Mailing

— Switched to mpark/wg21 pandoc format

— Add general description of span and mdspan

— Removed `mdspan_subspan` expo only type; use `basic_mdspan<`*see below*`>` instead

— Fixed typos in accessor table

— Made editorial changes to wording based on San Diego feedback

— Updated operational semantics subsection heading based on new style guidelines

1.10 P0009r9: Pre 2019-02-Kona Mailing

— Wording fixes based on guidance: LWG small group at 2018-11-SanDiego

1.11 P0009r8: Pre 2018-11-SanDiego Mailing

— Refinement based upon updated prototype / reference implementation

1.12 P0009r7: Post 2018-06-Rapperswil Mailing

— wording reworked based on guidance: LWG review at 2018-06-Rapperswil

— usage of `span` requires reference to C++20 working draft

— namespace for library TS `std::experimental::fundamentals_v3`

1.13 P0009r6 : Pre 2018-06-Rapperswil Mailing

P0009r5 was not taken up at 2018-03-Jacksonville meeting. Related LEWG review of P0900 at 2018-03-Jacksonville meeting

LEWG Poll We want the ability to customize the access to elements of span (ability to restrict, etc):

```
span<T, N, Accessor=...>
```

SF	F	N	A	SA
1	1	1	2	8

LEWG Poll We want the customization of `basic_mdspan` to be two concepts `Mapper` and `Accessor` (akin to `Allocator` design).

```
basic_mdspan<T, Extents, Mapper, Accessor>
mdspan<T, N...>
```

SF	F	N	A	SA
3	4	5	1	0

LEWG Poll: We want the customization of `basic_mdspan` to be an arbitrary (and potentially user-extensible) list of properties.

```
basic_mdspan<T, Extents, Properties...>
```

SF	F	N	A	SA
1	2	2	6	2

Changes from P0009r5 due to related LEWG reviews:

— Replaced variadic property list with *extents*, *layout mapping*, and *accessor* properties.

— Incorporated P0454r1.

  — Added accessor policy concept.

  — Renamed `mdspan` to `basic_mdspan`.

  — Added a `mdspan` alias to `basic_mdspan`.

1.14 P0009r5 : Pre 2018-03-Jacksonville Mailing

LEWG review of P0009r4 at 2017-11-Albuquerque meeting

LEWG Poll: We should be able to index with `span<int type[N]>` (in addition to array).

SF	F	N	A	SA
2	11	1	1	0

Against comment - there is not a proven needs for this feature.

LEWG Poll: We should be able to index with 1d `mdspan`.

SF	F	N	A	SA

0	8	7	0	0

LEWG Poll: We should put the requirement on “rank() <= N” back to “rank()==N”.

Unanimous consent

LEWG Poll: With the editorial changes from small group, plus the above polls, forward this to LWG for Fundamentals v3.

Unanimous consent

Changes from P0009r4:

— Removed nullptr constructor.

— Added constexpr to indexing operator.

— Indexing operator requires that `rank()==sizeof...(indices)`.

— Fixed typos in examples and moved them to appendix.

— Converted note on how extentions to access properties may cause reference to be a proxy type to an “see below” to make it normative.

1.15 P0009r4 : Pre 2017-11-Albuquerque Mailing

LEWG review at 2017-03-Kona meeting

LEWG review of P0546r1 at 2017-03-Kona meeting

LEWG Poll: Should we have a single template that covers both single and multi-dimensional spans?

SF	F	N	A	SA
1	6	2	6	3

Changes from P0009r3:

— Align with P0122r5 span proposal.

— Rename to `mdspan`, multidimensional span, to align with `span`.

— Move preferred array extents mechanism to appendix.

— Expose codomain as a `span`.

— Add layout mapping concept.

1.16 P0009r3 : Post 2016-06-Oulu Mailing

LEWG review at 2016-06-Oulu

LEWG did not like the name `array_ref`, and suggested the following alternatives: - `sci_span` - `numeric_span` - `multidimensional_span` - `multidim_span` - `mdspan` - `md_span` - `vla_span` - `multispan` - `multi_span`

LEWG Poll: Are member `begin()`/`end()` still good?

SF	F	N	A	SA
0	2	4	3	1

LEWG Poll: Want this proposal to provide range-producing functions outside `array_ref`?

SF	F	N	A	SA
0	1	3	2	3

LEWG Poll: Want a separate proposal to explore iteration design space?

SF	F	N	A	SA
9	1	0	0	0

Changes from P0009r2:

— Removed iterator support; a future paper will be written on the subject.

— Noted difference between multidimensional array versus language’s array-of-array-of-array…

— Clearly describe requirements for the embedded type aliases (`element_type`, `reference`, etc).

— Expanded description of how the variadic properties list would work.

— Stopped allowing `array_ref<T[N]>` in addition to `array_ref<extents<N>>`.

— Clarified domain, codomain, and domain -> codomain mapping specifications.

— Consistently use *extent* and *extents* for the multidimensional index space.

1.17 P0009r2 : Pre 2016-06-Oulu Mailing

LEWG review at 2016-02-Jacksonville.

Changes from P0009r1:

— Adding details for extensibility of layout mapping.

— Move motivation, examples, and relaxed incomplete array type proposal to separate papers.

  — P0331: Motivation and Examples for Polymorphic Multidimensional Array.

  — P0332: Relaxed Incomplete Multidimensional Array Type Declaration.

1.18 P0009r1 : Pre 2016-02-Jacksonville Mailing

LEWG review at 2015-10-Kona.

LEWG Poll: What should this feature be called?

Name	#
`view`	5
`span`	9
`array_ref`	6
`slice`	6
`array_view`	6
`ref`	0
`array_span`	7
`basic_span`	1
`object_span`	3
`field`	0

LEWG Poll: Do we want 0-length static extents?

SF	F	N	A	SA
3	4	2	3	0

LEWG POLL: Do we want the language to support syntaxes like `X[3][][][5]`?

Syntax	#
`view<int[3][0][][5], property1>`	12
`view<int, dimension<3, 0, dynamic_extent, 5>, property1>`	4
`view<int[3][0][dynamic_extent][5], property1>`	5
`view<int, 3, 0, dynamic_extent, 5, property1>`	4
`view<int, 3, 0, dynamic_extent, 5, properties<property1>>`	2
`view<arr<int, 3, 0, dynamic_extent, 5>, property1>`	4
`view<int[3][0][][5], properties<property1>>`	9

LEWG POLL: Do we want the variadic property list in template args (either raw or in `properties<>`)? Note there is no precedence for this in the library.

SF	F	N	A	SA
3	6	3	0	0

LEWG POLL: Do we want the per-view bounds-checking knob?

SF	F	N	A	SA
3	4	1	2	1

Changes from P0009r0:

— Renamed `view` to `array_ref`.

— How are users allowed to add properties? Needs elaboration in paper.

— `view<int[][][]>::layout` should be named.

— Rename `is_regular` (possibly to `is_affine`) to avoid overloading the term with the `Regular` concept.

— Make static span(), operator(), constructor, etc variadic.

— Demonstrate the need for improper access in the paper.

— In `operator()`, take integral types by value.

1.19 P0009r0 : Pre 2015-10-Kona Mailing

Original non-owning multidimensional array reference (`view`) paper with motivation, specification, and examples.

1.20 Related Activity

Related LEWG review of P0546r1 at 2017-11-Albuquerque meeting

LEWG Poll: `span` should specify the dynamic extent as the element type of the first template parameter rather than the (current) second template parameter

SF	F	N	A	SA
5	3	2	2	0

LEWG Poll: `span` should support the addition of access properties variadic template parameters

SF	F	N	A	SA
0	10	1	5	0

Authors agreed to bring a separate paper ([[P0900r0]]) discussing how the variadic properties will work.

2 Description

2.1 What we propose to add

This paper proposes adding to the C++ Standard Library a multidimensional array view, `mdspan`, along with classes, class templates, and constants for describing and creating multidimensional array views. It also proposes adding the `submdspan` function that "slices" (returns an `mdspan` that views a subset of) an existing `mdspan`.

The `mdspan` class template can represent arbitrary mixes of compile-time or run-time extents. Its element type can be any complete object type that is neither an abstract class type nor an array type. It has two customization opportunities for users: the *layout mapping* and the *accessor*. The layout mapping specifies the formula, and properties of the formula, for mapping a multidimensional index to an element of the array. The accessor governs how elements are read and written.

2.2 Definitions

A *multidimensional array view* views a multidimensional array, just as a `span` views a one-dimensional `array` or `vector`.

A *multidimensional array* of *rank* maps from a tuple of indices to a single offset index. Each of the indices in the tuple is in a bounded range whose inclusive lower bound is zero, and whose nonnegative exclusive upper bound is that index's *extent*. The array thus has extents. The offset index ranges over a subset of a bounded contiguous index range whose lower bound is zero, and whose upper bound is the product of the extents.

More formally, a multidimensional array of rank maps from its *domain*, a multidimensional index space of rank , to its *codomain*, a set of objects accessible from a contiguous range of integer indices. A *multidimensional index space* of rank is the Cartesian product $_0) \times [0, N_1) \times \ldots \times [0, N_{R-1})$ of half-open integer intervals, where the $_k$ for , ..., are the array's extents. A *multidimensional index* is a element of a multidimensional index space.

2.3 Why do we need multidimensional arrays?

Multidimensional arrays are fundamental concepts in many fields, including graphics, mathematics, statistics, engineering, and the sciences. Many programming languages thus come with multidimensional

array data structures either as a core language feature, or as a tightly integrated standard library. Example languages include Ada, ANSI Common Lisp, APL, C#, Fortran, Julia, Matlab, Mathematica, Pascal, Python (via NumPy), and Visual Basic. The original version of the Fortran language for the IBM 704 featured arrays with one, two, or three extents (Backus 1956, pp. 10-11).

Multidimensional arrays have long been useful for representing large amounts of data, describing points in physical space, or expressing approximations of functions. They are a natural way to represent mathematical objects like matrices and tensors. This makes multidimensional arrays a critical data structure for many computations at the heart of modern machine learning. In fact, one of the predominant machine learning frameworks is called TensorFlow.

2.4 Why are existing C++ data structures not enough?

C++ currently has the following approaches that could be used to represent multidimensional arrays:

1. "native" arrays where all the extents are compile-time constants, like `int[3][4][5]`;
2. pointer-of-pointers(-of-pointers...), like `int***`, set up as a data structure to view multidimensional data;
3. arrays-of-arrays(-of-arrays...) data structures, like `vector<vector<array<int, N>>>`; or
4. `gslice`, which selects a subset of indices of a `valarray` and can be used to impose a multidimensional array layout on the `valarray`, in a way analogous to `layout_stride`.

If a multidimensional array has any extents that are not known at compile time, Approach (1) does not work.

Approach (2) does not suffice as a stand-alone data structure, because a pointer-of-pointers does not carry along the array's run-time extents. Users thus end up building some subset of `mdspan`'s functionality to represent a multidimensional array view. Every run-time extent other than the rightmost requires a separate memory allocation for an array of pointers. A pointer-of-pointers also loses information about any dimensions known at compile time. Users cannot arbitrarily mix compile-time and run-time extents.

Approach (3) can mix `vector` and `array` to represent extents known at run time resp. compile time. However, any use of `vector` at any position other than the outermost results in the data structure no longer having a contiguous memory allocation (or a subset thereof) for the elements. This makes the data structure incompatible with many libraries that expect a subset of a contiguous allocation. Also, every run-time extent other than the rightmost requires a separate memory allocation for an array of arrays. In addition, each element access requires reading multiple memory locations ("pointer chasing"). Finally, the inlining depth for an element access is proportional to the array's rank.

Approach (4) is meant for addressing many elements of a `valarray` all at once. Even though `valarray` itself is a one-dimensional array, one can use `gslice` to make the `valarray` represent multidimensional data. Giving a `gslice` to `valarray::operator[]` returns something that references a subset of elements of the original `valarray`. However, the result (a `gslice_array` in the nonconst case, some type that might be an expression template in the const case) is not guaranteed to have an `operator[]`. Thus, it's not a view, whereas our proposed `submdspan` function always takes and returns a view. In the const case, the result might even be a (deep) copy of the input. Finally, `gslice` offers no efficient way to address a single element. The `gslice` constructor takes strides and lengths as `valarray`s and is meant for array-based computation. Accessing a single element requires accessing the memory of three `valarray`s.

2.5 Mixing compile-time and run-time extents

The fundamental reason to allow expressing extents at compile time is performance. Knowing an extent at compile time enables many compiler optimizations, such as unrolling and precomputing of offsets. These can significantly improve the generated code. Not storing extents at run time may help conserve registers and stack space.

In many fields, some extents are naturally known at compile time. For many physics and engineering algorithms, some extents are dictated by fundamental properties of the physical world or the discretization scheme. For example, the position of a particle in space requires a rank-3 array, since physical space has three dimensions. At the same time, other extents are only known at run time, such as the number of particles in a simulation. A natural data structure for storing a list of particles would thus be a rank-2 array, where the one run-time extent is the number of particles and the one compile-time extent is three. In graphics, some of the most fundamental objects are square matrices with 2, 3, or 4 rows and columns. The number of matrices with which one would like to compute might only be known at run time. This would make a rank-3 array with two compile-time extents a natural data structure for the matrices.

2.6 Why custom memory layouts?

Our `mdspan` class template permits custom layouts. Our proposal comes with three memory layouts:

— `layout_right`: C or C++ style, row major, where the rightmost index gives stride-1 access to the underlying memory;

— `layout_left`: Fortran or Matlab style, column major, where the leftmost index gives stride-1 access to the underlying memory;

— `layout_stride`: a generalization of the two layouts above, which stores a separate stride (possibly not one) for each extent.

"Custom" layouts besides these could include space-filling curves or "tiled" layouts.

An important reason we allow different layouts is language interoperability. For example, C++ and Fortran have different "native" layouts. Python's NumPy arrays have a configurable layout, to provide compatibility with both languages.

Control of the layout can also be used to write code that performs well on different computer architectures when only changing a template argument. Consider the following implementation of a parallel dense matrix-vector product.

```
using layout = /* see-below */;

std::mdspan<double, std::extents<int, N, M>, layout> A = ...;
std::mdspan<double, std::extents<int, N>> y = ...;
std::mdspan<double, std::extents<int, M>> x = ...;

std::ranges::iota_view range{0, N};

std::for_each(std::execution::par_unseq,
  std::ranges::begin(range), std::ranges::end(range),
  [=](int i) {
     double sum = 0.0;
     for(int j = 0; j < M; ++j) {
       sum += A[i, j] * x[j];
     }
     y[i] = sum;
  });
```

On conventional CPU architectures, this code performs well with `layout = layout_right`, the native C++ row-major layout. However, when offloading the `for_each` to NVIDIA GPUs (which NVIDIA's `nvc++` compiler can do), `layout = layout_left` (Fortran's column-major layout) performs much better, since it enables coalesced data access on the matrix `A`.

However, it is not enough to have just C++ and Fortran memory mappings. For instance, one way to compute tensor products is to decompose them into many matrix-matrix multiplications. The resulting decomposition may involve matrices with non-unit strides in both extents. This means that they have neither a row-major nor a column-major layout.

More complex layouts can improve performance significantly for some algorithms. For instance, tiling (a "matrix of small matrices" layout) can improve data locality for many computations relevant to linear algebra and the discretization of partial differential equations. Tiled layouts can also improve vectorization. For example, Intel's Math Kernel Library introduced the Vectorized Compact Routines. These provide "batched" matrix operations that increase available parallelism by operating on many matrices at once. The Vectorized Compact Routines accept matrices in an "interleaved" layout that optimizes vectorized memory access.

Another design goal for our custom layouts is to permit nonunique layouts. A *nonunique* layout lets multiple index tuples refer to the same element. This can save memory for data structures that have natural symmetry. For example, if `A` is a symmetric matrix, then `A[i, j]` and `A[j, i]` refer to the same element, so the element can and should only be stored once.

§ 2.7 Why custom accessors?

Custom accessors can provide information to the compiler, or permit the injection of special ways of doing data access. Most hardware today has more ways to access data than simple reads and writes. For example, some instructions affect caching behavior, by making loads and/or stores nontemporal (not cached at some level) or even noncoherent. Other instructions implement atomic access. This is why several of us proposed `atomic_ref`, as the heart of an "atomic accessor" for `mdspan`. C's `restrict` qualifier conveys whether an array is assumed never to alias another array in some context. The `volatile` keyword is yet another qualifier which limits compiler optimizations around data access. Custom `mdspan` accessors can apply `restrict` (if the C++ implementation supports this extension) or `volatile` to array accesses.

Custom accessors also address concerns relating to heterogeneous memory. Standard C++ does not have the idea of "memory spaces that normal code cannot access," but many extensions to C++ do have this idea. For example, a custom accessor could convey accessibility by CPU or GPU threads, so that the compiler would

prevent users from accessing GPU memory while running on the CPU, or vice versa. Multiple memory spaces occur in programming models other than for GPUs. For example, "partitioned global address space" models have a "global shared memory" that requires special operations to access. C++ libraries like Kokkos expose access to such memory using an analog of a custom accessor. Other accessors could expose an array interface to a persistent storage device that is not directly byte addressable. We do not propose such accessors here, but this is a customization point third-party libraries could directly use, and is available for any future extensions of the C++ standard for supporting heterogeneous memory.

For a discussion of the idea of accessors and several examples, please see (Keryell and Falcou 2016).

2.8 Subspan Support

A critical feature of this proposal is `submdspan`, the subspan or "slicing" function that returns a view of a subset of an existing `mdspan`. The result may have any rank up to and including the rank of the input. All of the aforementioned languages with multidimensional array support provide subspan capabilities. Subspans are important because they enable code reuse. For example, the inner loop in the dense matrix-vector product described above actually represents a *dot product* – an inner product of two vectors. If one already has a function for such an inner product, then a natural implementation would simply reuse that function. The LAPACK linear algebra library depends on subspan reuse for the performance of its one-sided "blocked" matrix factorizations (Cholesky, LU, and QR). These factorizations reuse textbook non-blocked algorithms by calling them on groups of contiguous columns at a time. This lets LAPACK spend as much time in dense matrix-matrix multiply (or algorithms with analogous performance) as possible.

However, due to possible time constraints, it was decided in May 2022 to move `submdspan` to its own paper. This would potentially allow mdspan to land C++23 even if there is not enough time to finish wording review on `submdspan`.

2.9 Why propose a multidimensional array view before a container?

Factoring views from containers generally makes sense. For example, one often sees functions that take `vector` by reference when they only need to access the `vector`'s elements or call `.size()` on it. This is one reason for `span`. Some of us have proposed a multidimensional array container, `mdarray` P1684, but we have focused on `mdspan` because we consider views more fundamental.

Many fields that compute with multidimensional arrays rely heavily on shared-memory parallel programming, where multiple processing units (threads, vector units, etc.) access different elements of the same array in parallel. Memory allocation and deallocation are "synchronization points" for parallel processing units, and thus hinder parallelization. This makes just *viewing* a multidimensional array, rather than managing its ownership, the most fundamental way for parallel computations to express how they access an array.

It is often necessary to view previously allocated memory as a multidimensional array. An important special case is when C++ code is calling or being called from another programming language, such as C, Fortran, or Python. This use case matters enough to Python that its C API defines a Buffer Protocol for viewing multidimensional arrays across languages. Language interoperability is key to the success of the various Python-based data analysis frameworks built up around NumPy.

2.10 Use multiple-parameter operator[] for array access

We welcome multiple-parameter `operator[]` as the preferred multidimensional array access operator. P1161R3, now part of C++20, prepared the way for this by deprecating comma expressions inside `operator[]` invocations. P2128R6, which proposed changing `operator[]` to accept multiple parameters, was approved at the October 2021 WG21 Plenary meeting. Please refer to P2128 for an extensive discussion.

Many existing libraries use the function call `operator()` for multidimensional array access, with `operator[]` available for rank-1 (single-dimensional) `mdspan`. P2128 gives examples. It's straightforward to adapt these libraries to transition to `mdspan`. For example, a subclass or wrapper of `mdspan` can provide an `operator()` that simply forwards to `mdspan::operator[]`. The subclass or wrapper can then deprecate `operator()` to help developers find and change all the code that uses it.

2.11 Reference Implementation

A reference implementation of this proposal under BSD license is available at: mdspan. This implementation is also available on godbolt for experimentation: godbolt.

2.12 References

— J. W. Backus et al. “Programmer’s Reference Manual: Fortran Automatic Coding System for the IBM 704.” Applied Science Division and Programming Research Department, International Business Machines Corporation, Oct. 15, 1956. Available online (last accessed Oct. 10, 2021).

— D. Hollman, C. Trott, M. Hoemmen, and D. Sunderland. “`mdarray`: An Owning Multidimensional Array Analog of `mdspan`.” P1684r0, May 28, 2019. Available online (last accessed Oct. 10, 2021).

— R. Keryell and J. Falcou. “Accessors: A C++ standard library class to qualify data accesses.” P0367r0, May 29, 2016. Available online (last accessed Oct. 10, 2021).

§ 3 Editing Notes

The proposed changes are relative to the working draft of the standard as of N4842.

The ﷐ character is used to denote a placeholder section number, table number, or paragraph number which the editor shall determine.

Add the header `<mdspan>` to the “C++ library headers” table in **[headers]** in a place that respects the table’s current alphabetic order.

Add the header `<mdspan>` to the “Containers library summary” table in **[containers.general]** below the listing for `<span>`.

§ 4 Wording

The ﷐ character is used to denote a placeholder subclause number which the editor shall determine.

*In **[version.syn]**, add:*

```
#define __cpp_lib_mdspan YYYYMML // also in <mdspan>
```

1 Adjust the placeholder value as needed so as to denote this proposal’s date of adoption.

*Make the following changes to 24.7.1 **[views.general]**,*

2 The header `<span>` defines the view span. The header `<mdspan>` defines the class template `mdspan` and other facilities for interacting with these multidimensional views.

---

*Add the following subclauses to the end of the **[views]** subclause (after `span`):*

24.7.﷐ Header `<mdspan>` synopsis [mdspan.syn]

```
namespace std {
  // [mdspan.extents], class template extents
  template<class SizeType, size_t... Extents>
    class extents;

  template<class SizeType, size_t Rank>
    using dextents = see below;

  // [mdspan.layout], Layout mapping policies
  struct layout_left;
  struct layout_right;
  struct layout_stride;

  // [mdspan.accessor.default]
  template<class ElementType>
    class default_accessor;

  // [mdspan.mdspan], class template mdspan
  template<class ElementType, class Extents, class LayoutPolicy = layout_right,
           class AccessorPolicy = default_accessor<ElementType>>
    class mdspan;
}
```

24.7.﷐ Overview [mdspan.terms]

1 A *multidimensional index space* is a Cartesian product of integer intervals. Each interval can be represented by a half-open range $[L_i, U_i)$, where L_i and U_i are the lower and upper bounds of the i^{th} dimension. The *rank* of a multidimensional index space is the number of intervals it represents. The *size of a multidimensional index space* is the product of $U_i - L_i$ for each dimension i if its rank is greater than 0, and 1 otherwise.

2 A pack of integers `idx` is a *multidimensional index* in a multidimensional index space S (or representation thereof) if both of following are true:

(2.1) — `sizeof...(idx)` is equal to rank of S, and

(2.2) — For all i in the range [0,rank), the i^{th} value of `idx` is an integer in the interval $[L_i, U_i)$ of S.

3 An integer r is a *rank index* of an index space S if r is in the range [0, rank).

24.7.☐ Class template `extents` [mdspan.extents]

24.7.☐.1 Overview [mdspan.extents.overview]

1 The class template `extents` represents a multidimensional index space of rank equal to `sizeof...(Extents)`. In subclause 24.7, `extents` will be used synonymously with multidimensional index space.

```
namespace std {

template<class SizeType, size_t... Extents>
class extents {
public:
  using size_type = SizeType;
  using rank_type = size_t;

  // [mdspan.extents.obs], Observers of the multidimensional index space
  static constexpr rank_type rank() noexcept { return sizeof...(Extents); }
  static constexpr rank_type rank_dynamic() noexcept { return dynamic-index(rank()); }
  static constexpr size_t static_extent(rank_type) noexcept;
  constexpr size_type extent(rank_type) const noexcept;


  // [mdspan.extents.ctor], Constructors
  constexpr extents() noexcept = default;

  template<class OtherSizeType, size_t... OtherExtents>
    explicit(see below)
    constexpr extents(const extents<OtherSizeType, OtherExtents...>&) noexcept;
  template<class... OtherSizeTypes>
    explicit constexpr extents(OtherSizeTypes...) noexcept;
  template<class OtherSizeType, size_t N>
    explicit(N != rank_dynamic())
    constexpr extents(span<OtherSizeType, N>) noexcept;
  template<class OtherSizeType, size_t N>
    explicit(N != rank_dynamic())
    constexpr extents(const array<OtherSizeType, N>&) noexcept;

  // [mdspan.extents.cmp], extents comparison operators
  template<class OtherSizeType, size_t... OtherExtents>
    friend constexpr bool operator==(const extents&, const extents<OtherSizeType, OtherExtents...>&)
        noexcept;

  // [mdspan.extents.helpers], exposition only helpers
  constexpr size_t fwd-prod-of-extents(rank_type) const noexcept; // exposition only
  constexpr size_t rev-prod-of-extents(rank_type) const noexcept; // exposition only
  template<class OtherSizeType>
  static constexpr auto index-cast(OtherSizeType&&) noexcept; // exposition only

private:
  static constexpr rank_type dynamic-index(rank_type) noexcept; // exposition only
  static constexpr rank_type dynamic-index-inv(rank_type) noexcept; // exposition only

  array<size_type, rank_dynamic()> dynamic-extents{}; // exposition only
};

template <class... Integrals>
explicit extents(Integrals...)
  -> see below;

}
```

2 *Mandates:*

(2.1) — `SizeType` is a signed or unsigned integer type, and

(2.2) — each element of `Extents` is either equal to `dynamic_extent`, or is representable as a value of type `SizeType`.

3 Each specialization of `extents` models `regular` and is trivially copyable.

4 Let E_r be the r^{th} element of `Extents`. E_r is a *dynamic extent* if it is equal to `dynamic_extent`, otherwise E_r is a static extent. Let D_r be the value of *`dynamic-extents[dynamic-index(r)]`* if E_r is a dynamic extent, otherwise E_r.

5 The r^{th} interval of the multidimensional index space represented by an `extents` object is $[0, D_r)$.

24.7.🔲.2 Exposition-only helpers [mdspan.extents.helpers]

```
static constexpr rank_type dynamic-index(rank_type i) noexcept; // exposition only
```

1 *Precondition:* `i <= rank()` is `true`.

2 *Returns:* Number of E_r with $r < i$ for which E_r is a dynamic extent.

```
static constexpr rank_type dynamic-index-inv(rank_type i) noexcept; // exposition only
```

3 *Precondition:* `i < rank_dynamic()` is `true`.

4 *Returns:* Minimum value of r such that *dynamic-index*(r+1) `==` `i+1` is `true`.

```
constexpr size_t fwd-prod-of-extents(rank_type i) const noexcept; // exposition only
```

5 *Precondition:* `i <= rank()` is `true`.

6 *Returns:* If `i > 0` is `true`, the product of `extent(`k`)` for all k in the range [0, `i`), otherwise `1`.

```
constexpr size_t rev-prod-of-extents(rank_type i) const noexcept; // exposition only
```

7 *Precondition:* `i < rank()` is `true`.

8 *Returns:* If `i+1 < rank()` is `true`, the product of `extent(`k`)` for all k in the range [`i+1` , `e.rank()`), otherwise `1`.

```
  template<class OtherSizeType>
  static constexpr auto index-cast(OtherSizeType&& i) noexcept; // exposition only
```

9 *Effects:*

(9.1) — If `OtherSizeType` is an integral type other than `bool`, then equivalent to `return i;`, otherwise

(9.2) — equivalent to `return static_cast<size_type>(i);`. *[Note:* This function will always return an integral type other than `bool`. Since this function's call sites are constrained on convertibility of `OtherSizeType` to `size_type`, integer-class types can use the `static_cast` branch without loss of precision.— *end note]*

24.7.🔲.3 Constructors [mdspan.extents.ctor]

```
template<class OtherSizeType, size_t... OtherExtents>
  explicit(see below)
  constexpr extents(const extents<OtherSizeType, OtherExtents...>& other) noexcept;
```

1 *Constraints:*

(1.1) — `sizeof...(OtherExtents) == rank()` is `true`.

(1.2) — `((OtherExtents == dynamic_extent || Extents == dynamic_extent || OtherExtents == Extents) && ...)` is `true`.

2 *Preconditions:*

(2.1) — `other.extent(`r`)` equals E_r for each r for which E_r is a static extent, and

(2.2) — either

  — `sizeof...(OtherExtents)` is zero, or

  — `other.extent(r)` is representable as a value of type `size_type` for all rank index `r` of `other`.

3 *Postconditions:* `*this == other` is `true`.

4 *Remarks:* The expression inside `explicit` is equivalent to:

```
(((Extents!=dynamic_extent) && (OtherExtents==dynamic_extent)) || ... ) ||
(numeric_limits<size_type>::max() < numeric_limits<OtherSizeType>::max())
```

```
template<class... OtherSizeTypes>
  explicit constexpr extents(OtherSizeTypes... exts) noexcept;
```

5 *Constraints:*

(5.1) — `(is_convertible_v<OtherSizeTypes, size_type> && ...)` is `true`,

(5.2) — `(is_nothrow_constructible_v<size_type, OtherSizeTypes> && ...)` is `true`, and

(5.3) — `sizeof...(OtherSizeTypes) == rank_dynamic() || sizeof...(OtherSizeTypes) == rank()` is `true`. *[Note:* One

can construct `extents` from just dynamic extents, which are all the values getting stored, or from all the extents with a precondition. — *end note]*

6 Let `exts_arr` be `array<size_type, sizeof...(OtherSizeTypes)>{static_cast<size_type>(std::move(exts))...}`

7 *Preconditions:*

(7.1) — If `sizeof...(OtherSizeTypes) != rank_dynamic()` is `true`, `exts_arr[r]` equals E_r for each r for which E_r is a static extent, and

(7.2) — either

— `sizeof...(exts) == 0` is `true`, or

— each element of `exts` is nonnegative and is representable as a value of type `size_type`.

8 *Postconditions:* `*this == extents(exts_arr)` is `true`.

```
template<class OtherSizeType, size_t N>
  explicit(N != rank_dynamic())
  constexpr extents(span<OtherSizeType, N> exts) noexcept;
template<class OtherSizeType, size_t N>
  explicit(N != rank_dynamic())
  constexpr extents(const array<OtherSizeType, N>& exts) noexcept;
```

9 *Constraints:*

(9.1) — `is_convertible_v<const OtherSizeType&, size_type>` is `true`,

(9.2) — `is_nothrow_constructible_v<size_type, const OtherSizeType&>` is `true`, and

(9.3) — `N==rank_dynamic() || N==rank()` is `true`.

10 *Preconditions*:

(10.1) — If `N != rank_dynamic()` is `true`, `exts[r]` equals E_r for each r for which E_r is a static extent, and

(10.2) — either

— `N` is zero, or

— `exts[r]` is nonnegative and is representable as a value of type `size_type` for all rank index `r`.

11 *Effects:*

(11.1) — If `N` equals `dynamic_rank()`, for all d in the range [0, `rank_dynamic()`), direct-non-list-initializes *dynamic-extent*[d] with `as_const(exts[`d`])`.

(11.2) — Otherwise, for all d in the range [0, `rank_dynamic()`), direct-non-list-initializes *dynamic-extent*[d] with `as_const(exts[`*dynamic-index-inv*(d)`])`.

```
template <class... Integrals>
explicit extents(Integrals...) -> see below;
```

12 *Constraints:* `(is_convertible_v<Integrals, size_t> && ...)` is `true`.

13 *Remarks:* The deduced type is `dextents<size_t, sizeof...(Integrals)>`.

24.7.﷿.4 Observers of the multidimensional index space [mdspan.extents.obs]

```
static constexpr size_t static_extent(rank_type i) noexcept;
```

1 *Preconditions:* `i < rank()` is `true`.

2 *Returns:* E_i.

```
constexpr size_type extent(rank_type i) const noexcept;
```

3 *Preconditions:* `i < rank()` is `true`.

4 *Returns:* D_i.

24.7.﷿.5 Comparison operators [mdspan.extents.cmp]

```
template<class OtherSizeType, size_t... OtherExtents>
```

```
  friend constexpr bool operator==(const extents& lhs,
                                   const extents<OtherSizeType, OtherExtents...>& rhs) noexcept;
```

1 *Returns:* `true` if `lhs.rank()` equals `rhs.rank()` and if `lhs.extent(r)` equals `rhs.extent(r)` for every rank index `r` of `rhs`, otherwise `false`.

24.7.🔲.6 Template alias `dextents` [mdspan.extents.dextents]

```
template <class SizeType, size_t Rank>
  using dextents = see below;
```

1 *Result:* A type `E` that is a specialization of `extents` such that `E::rank() == Rank && E::rank() == E::rank_dynamic()` is `true`, and `E::size_type` denotes `SizeType`.

24.7.🔲 Layout mapping [mdspan.layout]

24.7.🔲.1 General [mdspan.layout.general]

1 In subclause 24.7.🔲.2 and subclause 24.7.🔲.3

(1.1) — `M` denotes a layout mapping class.

(1.2) — `m` denotes a (possibly const) value of type `M`.

(1.3) — `i` and `j` are packs of (possibly const) integers which are multidimensional indices in `m.extents()` ([mdspan.terms]). *[Note:* The type of each element in multidimensional indices can be a different integer type. — *end note]*

(1.4) — `r` is a (possibly const) rank index of `typename M::extents_type`.

(1.5) — d_r is a pack of (possibly const) integers for which `sizeof...(`d_r`) == M::extents_type::rank()` is `true`, the r-th element is equal to `1`, and all other elements are equal to `0`.

2 In subclauses from [mdspan.layout.reqmts] to [mdspan.layoutstride] let *`is-mapping-of`* be the variable template defined as follows

```
template<class Layout, class Mapping>
constexpr bool is-mapping-of =
  is_same_v<typename Layout::template mapping<typename Mapping::extents_type>, Mapping>;
```

24.7.🔲.2 Requirements [mdspan.layout.reqmts]

1 A type `M` meets the *layout mapping* requirements if:

(1.1) — `M` models `copyable` and `equality_comparable`,

(1.2) — `is_nothrow_move_constructible_v<M>` is `true`,

(1.3) — `is_nothrow_move_assignable_v<M>` is `true`,

(1.4) — `is_nothrow_swappable_v<M>` is `true`, and

(1.5) — the following types and expressions are well-formed and have the specified semantics.

```
typename M::extents_type
```

2 *Result:* A type which is a specialization of `extents`.

```
typename M::size_type
```

3 *Result:* `typename M::extents_type::size_type`.

```
typename M::rank_type
```

4 *Result:* `typename M::extents_type::rank_type`.

```
typename M::layout_type
```

5 *Result:* A type `MP` which meets the layout mapping policy requirements ([mdspan.layoutpolicy.reqmts]) and for which *`is-mapping-of`*`<MP, M>` is `true`.

```
m.extents()
```

6 *Result:* `const typename M::extents_type&`

```
m(i...)
```

Result: `typename M::size_type`

Returns: A nonnegative integer less than `numeric_limits<typename M::size_type>::max()` and less than or equal to `numeric_limits<size_t>::max()`.

```
m(i...) == m(static_cast<typename M::size_type>(i)...)
```

Result: `bool`

Value: `true`

```
m.required_span_size()
```

Result: `typename M::size_type`

Returns: If the size of the multidimensional index space `m.extents()` is 0, then `0`, else `1` plus the maximum value of `m(i...)` for all `i`.

```
m.is_unique()
```

Result: `bool`

Returns: `true` only if for every `i` and `j` where `(i != j || ...)` is `true`, `m(i...) != m(j...)` is `true`. *[Note:* A mapping may return `false` even if the condition is met. For certain layouts it may not be feasible to determine efficiently whether the layout is unique.— *end note]*

```
m.is_contiguous()
```

Result: `bool`

Returns: `true` only if for all k in the range [0, `m.required_span_size()`) there exists an `i` such that `m(i...)` equals k. *[Note:* A mapping may return `false` even if the condition is met. For certain layouts it may not be feasible to determine efficiently whether the layout is contiguous.— *end note]*

```
m.is_strided()
```

Result: `bool`

Returns: `true` only if for every rank index r of `m.extents()` there exists an integer s_r such that, for all `i` where `(i+`d_r`)` is a multidimensional index in `m.extents()` ([mdspan.terms]), `m((i+`d_r`)...) - m(i...)` equals s_r. *[Note:* This implies that for a strided layout `m(`$i_0, ..., i_k$`) = m(`$0, ..., 0$`)` + i_0 * s_0 + ... + i_k * s_k. — *end note] [Note:* A mapping may return `false` even if the condition is met. For certain layouts it may not be feasible to determine efficiently whether the layout is strided.— *end note]*

```
m.stride(r)
```

Preconditions: `m.is_strided()` is `true`.

Result: `typename M::size_type`

Returns: s_r as defined in `m.is_strided()` above.

```
M::is_always_unique()
```

Result: A constant expression ([expr.const]) of type `bool`.

Returns: `true` only if `m.is_unique()` is `true` for all possible objects `m` of type `M`. *[Note:* A mapping may return `false` even if the above condition is met. For certain layout mappings it may not be feasible to determine whether every instance is unique.— *end note]*

```
M::is_always_contiguous()
```

Result: A constant expression ([expr.const]) of type `bool`.

Returns: `true` only if `m.is_contiguous()` is `true` for all possible objects `m` of type `M`. *[Note:* A mapping may return `false` even if the above condition is met. For certain layout mappings it may not be feasible to determine whether every instance is contiguous.— *end note]*

```
M::is_always_strided()
```

Result: A constant expression ([expr.const]) of type `bool`.

Returns: `true` only if `m.is_strided()` is `true` for all possible objects `m` of type `M`. *[Note:* A mapping may return `false` even if the above condition is met. For certain layout mappings it may not be feasible to determine

whether every instance is strided.— *end note]*

24.7.﷿.3 Layout mapping policy requirements [mdspan.layoutpolicy.reqmts]

1 A type `MP` meets the *layout mapping policy* requirements if for a type `E` that is a specialization of `extents`, `MP::mapping<E>` is valid and denotes a type `X` that meets the layout mapping requirements ([mdspan.layout.reqmts]), and for which the *qualified-id* `X::layout_type` is valid and denotes the type `MP` and the *qualified-id* `X::extents_type` denotes `E`.

24.7.﷿.4 Layout mapping policies [mdspan.layoutpolicy.overview]

```
namespace std {

struct layout_left {
  template<class Extents>
    class mapping;
};
struct layout_right {
  template<class Extents>
    class mapping;
};
struct layout_stride {
  template<class Extents>
    class mapping;
};

}
```

1 Each of `layout_left`, `layout_right`, and `layout_stride` meets the layout mapping policy requirements and is a trivial type.

24.7.﷿.5 Class template `layout_left::mapping` [mdspan.layoutleft]

24.7.﷿.5.1 Overview [mdspan.layoutleft.overview]

1 `layout_left` provides a layout mapping where the leftmost extent has stride 1, and strides increase left-to-right as the product of extents.

```
namespace std {

template<class Extents>
class layout_left::mapping {
  public:
    using extents_type = Extents;
    using size_type = typename extents_type::size_type;
    using rank_type = typename extents_type::rank_type;
    using layout_type = layout_left;

    // [mdspan.layoutleft.ctor], Constructors
    constexpr mapping() noexcept = default;
    constexpr mapping(const mapping&) noexcept = default;
    constexpr mapping(const extents_type&) noexcept;
    template<class OtherExtents>
      explicit(!is_convertible_v<OtherExtents, extents_type>)
      constexpr mapping(const mapping<OtherExtents>&) noexcept;
    template<class OtherExtents>
      explicit(see below)
      constexpr mapping(const layout_right::mapping<OtherExtents>&) noexcept;
    template<class OtherExtents>
      explicit(extents_type::rank() > 0)
      constexpr mapping(const layout_stride::mapping<OtherExtents>&);

    constexpr mapping& operator=(const mapping&) noexcept = default;

    // [mdspan.layoutleft.obs], Observers
    constexpr const extents_type& extents() const noexcept { return extents_; }

    constexpr size_type required_span_size() const noexcept;

    template<class... Indices>
      constexpr size_type operator()(Indices...) const noexcept;

    static constexpr bool is_always_unique() noexcept { return true; }
    static constexpr bool is_always_contiguous() noexcept { return true; }
    static constexpr bool is_always_strided() noexcept { return true; }

    static constexpr bool is_unique() noexcept { return true; }
    static constexpr bool is_contiguous() noexcept { return true; }
```

```
    static constexpr bool is_strided() noexcept { return true; }

    constexpr size_type stride(rank_type) const noexcept;

    template<class OtherExtents>
      friend constexpr bool operator==(const mapping&, const mapping<OtherExtents>&) noexcept;

  private:
    extents_type extents_{}; // exposition only
};

}
```

2 If `Extents` is not a specialization of `extents`, then the program is ill-formed.

3 `layout_left::mapping<E>` is a trivially copyable type that models `regular` for each `E`.

24.7.▯.5.2 Constructors [mdspan.layoutleft.ctor]

```
constexpr mapping(const extents_type& e) noexcept;
```

1 *Preconditions:* The size of the multidimensional index space `e` is representable as a value of type `size_type` ([basic.fundamental]).

2 *Effects:* Direct-non-list-initializes *extents_* with `e`.

```
template<class OtherExtents>
  explicit(!is_convertible_v<OtherExtents, extents_type>)
  constexpr mapping(const mapping<OtherExtents>& other) noexcept;
```

3 *Constraints:* `is_constructible_v<extents_type, OtherExtents>` is `true`.

4 *Preconditions*: `other.required_span_size()` is representable as a value of type `size_type` ([basic.fundamental]).

5 *Effects:* Direct-non-list-initializes *extents_* with `other.extents()`.

```
template<class OtherExents>
  explicit(!is_convertible_v<OtherExtents, extents_type>)
  constexpr mapping(const layout_right::mapping<OtherExtents>& other) noexcept;
```

6 *Constraints:*

(6.1) — `extents_type::rank() <= 1` is `true`, and

(6.2) — `is_constructible_v<extents_type, OtherExtents>` is `true`.

7 *Preconditions*: `other.required_span_size()` is representable as a value of type `size_type` ([basic.fundamental]).

8 *Effects:* Direct-non-list-initializes *extents_* with `other.extents()`.

```
template<class OtherExtents>
  explicit(extents_type::rank() > 0)
  constexpr mapping(const layout_stride::mapping<OtherExtents>& other);
```

9 *Constraints:* `is_constructible_v<extents_type, OtherExtents>` is `true`.

10 *Preconditions:*

(10.1) — If `extents_type::rank() > 0` is `true`, then for all r in the range [0, `extents_type::rank()`), `other.stride(`r`)` equals `extents().`*`fwd-prod-of-extents`*`(`r`)`, and

(10.2) — `other.required_span_size()` is representable as a value of type `size_type` ([basic.fundamental]).

11 *Effects:* Direct-non-list-initializes *extents_* with `other.extents()`.

24.7.▯.5.3 Observers [mdspan.layoutleft.obs]

```
constexpr size_type required_span_size() const noexcept;
```

1 *Returns:* `extents().`*`fwd-prod-of-extents`*`(extents_type::rank())`.

```
template<class... Indices>
  constexpr size_type operator()(Indices... i) const noexcept;
```

2 *Constraints:*

(2.1) — `sizeof...(Indices) == extents_type::rank()` is `true`,

(2.2) — `(is_convertible_v<Indices, size_type> && ...)` is `true`, and

(2.3) — `(is_nothrow_constructible_v<size_type, Indices> && ...)` is `true`.

3 *Preconditions:* `extents_type::`*`index-cast`*`(i)` is a multidimensional index in *`extents_`* ([mdspan.terms]).

4 *Effects:* Let `P` be a parameter pack such that `is_same_v<index_sequence_for<Indices...>, index_sequence<P...>>` is `true`.
Equivalent to: `return ((static_cast<size_type>(i)*stride(P)) + ... + 0);`

```
constexpr size_type stride(rank_type i) const;
```

5 *Constraints:* `extents_type::rank() > 0` is `true`.

6 *Preconditions:* `i < extents_type::rank()` is `true`.

7 *Returns:* `extents().`*`fwd-prod-of-extents`*`(i)`.

```
template<class OtherExtents>
  friend constexpr bool operator==(const mapping& x, const mapping<OtherExtents>& y) noexcept;
```

8 *Constraints:* `extents_type::rank() == OtherExtents::rank()` is `true`.

9 *Effects:* Equivalent to: `return x.extents() == y.extents();`

24.7.❑.6 Class template `layout_right::mapping` [mdspan.layoutright]

24.7.❑.6.1 Overview [mdspan.layoutright.overview]

1 `layout_right` provides a layout mapping where the rightmost extent is stride 1, and strides increase right-to-left as the product of extents.

```
namespace std {

template<class Extents>
class layout_right::mapping {
  public:
    using extents_type = Extents;
    using size_type = typename extents_type::size_type;
    using rank_type = typename extents_type::rank_type;
    using layout_type = layout_right;

    // [mdspan.layoutright.ctor], Constructors
    constexpr mapping() noexcept = default;
    constexpr mapping(const mapping&) noexcept = default;
    constexpr mapping(const extents_type&) noexcept;
    template<class OtherExtents>
      explicit(!is_convertible_v<OtherExtents, extents_type>)
      constexpr mapping(const mapping<OtherExtents>&) noexcept;
    template<class OtherExtents>
      explicit(see below)
      constexpr mapping(const layout_left::mapping<OtherExtents>&) noexcept;
    template<class OtherExtents>
      explicit(extents_type::rank() > 0)
      constexpr mapping(const layout_stride::mapping<OtherExtents>&) noexcept;

    constexpr mapping& operator=(const mapping&) noexcept = default;

    // [mdspan.layoutright.obs], Observers
    constexpr const extents_type& extents() const noexcept { return extents_; }

    constexpr size_type required_span_size() const noexcept;

    template<class... Indices>
      constexpr size_type operator()(Indices...) const noexcept;

    static constexpr bool is_always_unique() noexcept { return true; }
    static constexpr bool is_always_contiguous() noexcept { return true; }
    static constexpr bool is_always_strided() noexcept { return true; }

    static constexpr bool is_unique() noexcept { return true; }
    static constexpr bool is_contiguous() noexcept { return true; }
    static constexpr bool is_strided() noexcept { return true; }

    constexpr size_type stride(rank_type) const noexcept;

    template<class OtherExtents>
      friend constexpr bool operator==(const mapping&, const mapping<OtherExtents>&) noexcept;

  private:
    extents_type extents_{}; // exposition only
```

```
};
}
```

2 If `Extents` is not a specialization of `extents`, then the program is ill-formed.

3 `layout_right::mapping<E>` is a trivially copyable type that models `regular` for each `E`.

24.7.[U+FFFD].6.2 Constructors [mdspan.layoutright.ctor]

```
constexpr mapping(const extents_type& e) noexcept;
```

1 *Preconditions:* The size of the multidimensional index space `e` is representable as a value of type `size_type` ([basic.fundamental]).

2 *Effects:* Direct-non-list-initializes *`extents_`* with `e`.

```
template<class OtherExtents>
  explicit(!is_convertible_v<OtherExtents, extents_type>)
  constexpr mapping(const mapping<OtherExtents>& other) noexcept;
```

3 *Constraints:* `is_constructible_v<extents_type, OtherExtents>` is `true`.

4 *Preconditions*: `other.required_span_size()` is representable as a value of type `size_type` ([basic.fundamental]).

5 *Effects:* Direct-non-list-initializes *`extents_`* with `other.extents()`.

```
template<class OtherExtents>
  explicit(!is_convertible_v<OtherExtents, extents_type>)
  constexpr mapping(const layout_left::mapping<OtherExtents>& other) noexcept;
```

6 *Constraints:*

(6.1) — `extents_type::rank() <= 1` is `true`, and

(6.2) — `is_constructible_v<extents_type, OtherExtents>` is `true`.

7 *Preconditions*: `other.required_span_size()` is representable as a value of type `size_type` ([basic.fundamental]).

8 *Effects:* Direct-non-list-initializes *`extents_`* with `other.extents()`.

```
template<class OtherExtents>
  explicit(extents_type::rank() > 0)
  constexpr mapping(const layout_stride::mapping<OtherExtents>& other) noexcept;
```

9 *Constraints:* `is_constructible_v<extents_type, OtherExtents>` is `true`.

10 *Preconditions:*

(10.1) — If `extents_type::rank() > 0` is `true`, then for all *r* in the range [0, `extents_type::rank()`), `other.stride(`*r*`)` equals `extents().`*`rev-prod-of-extents`*`(`*r*`)`.

(10.2) — `other.required_span_size()` is representable as a value of type `size_type` ([basic.fundamental]).

11 *Effects:* Direct-non-list-initializes *`extents_`* with `other.extents()`.

24.7.[U+FFFD].6.3 Observers [mdspan.layoutright.obs]

```
size_type required_span_size() const noexcept;
```

1 *Returns:* `extents().`*`fwd-prod-of-extents`*`(extents_type::rank())`

```
template<class... Indices>
  constexpr size_type operator()(Indices... i) const noexcept;
```

2 *Constraints:*

(2.1) — `sizeof...(Indices) == extents_type::rank()` is `true`, and

(2.2) — `(is_convertible_v<Indices, size_type> && ...)` is `true`.

(2.3) — `(is_nothrow_constructible_v<size_type, Indices> && ...)` is `true`.

3 *Preconditions:* `extents_type::`*`index-cast`*`(i)` is a multidimensional index in *`extents_`* ([mdspan.terms]).

4 *Effects:* Let `P` be a parameter pack such that `is_same_v<index_sequence_for<Indices...>, index_sequence<P...>>` is `true`.
Equivalent to: `return ((static_cast<size_type>(i)*stride(P)) + ... + 0);`

```
constexpr size_type stride(rank_type i) const noexcept;
```

5 *Constraints:* `extents_type::rank() > 0` is `true`.

6 *Preconditions:* `i < extents_type::rank()` is `true`.

7 *Returns:* `extents().`*`rev-prod-of-extents`*`(i)`

```
template<class OtherExtents>
  friend constexpr bool operator==(const mapping& x, const mapping<OtherExtents>& y) noexcept;
```

8 *Constraints:* `extents_type::rank() == OtherExtents::rank()` is `true`.

9 *Effects:* Equivalent to: `return x.extents() == y.extents();`

24.7.🔲.7 Class template `layout_stride::mapping` [mdspan.layoutstride]

24.7.🔲.7.1 Overview [mdspan.layoutstride.overview]

1 `layout_stride` provides a layout mapping where the strides are user-defined.

```
namespace std {

template<class Extents>
class layout_stride::mapping {
  public:
    using extents_type = Extents;
    using size_type = typename extents_type::size_type;
    using rank_type = typename extents_type::rank_type;
    using layout_type = layout_stride;

  private:
    static constexpr rank_type rank_ = extents_type::rank(); // exposition only

  public:
    // [mdspan.layoutstride.ctor], Constructors
    constexpr mapping() noexcept = default;
    constexpr mapping(const mapping&) noexcept = default;
    template<class OtherSizeType>
    constexpr mapping(const extents_type&,
                      span<OtherSizeType, rank_>) noexcept;
    template<class OtherSizeType>
    constexpr mapping(const extents_type&,
                      const array<OtherSizeType, rank_>&) noexcept;

    template<class StridedLayoutMapping>
      explicit(see below)
      constexpr mapping(const StridedLayoutMapping&) noexcept;

    constexpr mapping& operator=(const mapping&) noexcept = default;

    // [mdspan.layoutstride.obs], Observers
    constexpr const extents_type& extents() const noexcept { return extents_; }
    constexpr array<size_type, rank_> strides() const noexcept
    { return strides_; }

    constexpr size_type required_span_size() const noexcept;

    template<class... Indices>
      constexpr size_type operator()(Indices...) const noexcept;

    static constexpr bool is_always_unique() noexcept { return true; }
    static constexpr bool is_always_contiguous() noexcept { return false; }
    static constexpr bool is_always_strided() noexcept { return true; }

    static constexpr bool is_unique() noexcept { return true; }
    constexpr bool is_contiguous() const noexcept;
    static constexpr bool is_strided() noexcept { return true; }

    constexpr size_type stride(rank_type i) const noexcept { return strides_[i]; }

    template<class OtherMapping>
      friend constexpr bool operator==(const mapping&, const OtherMapping&) noexcept;

  private:
    extents_type extents_{}; // exposition only
    array<size_type, rank_> strides_{}; // exposition only
};
}
```

2 If `Extents` is not a specialization of `extents`, then the program is ill-formed.

3 `layout_stride::mapping<E>` is a trivially copyable type that models `regular` for each `E`.

24.7.𝔰.7.2 Exposition-only helpers [mdspan.layoutstride.expo]

1 Let *REQUIRED-SPAN-SIZE*(`e, strides`) be:

(1.1) — 1, if `e.rank() == 0` is `true`, otherwise

(1.2) — 0, if the size of the multidimensional index space `e` is 0, otherwise

(1.3) — 1 plus the sum of products of `(e.extent(`r`) - 1)` and `(strides[`r`])` for all r in the range [0, `e.rank()`).

2 Let *OFFSET*(`m`) be:

(2.1) — `m()`, if `e.rank() == 0` is `true`, otherwise

(2.2) — 0, if the size of the multidimensional index space `e` is 0, otherwise

(2.3) — `m(z...)` for a pack of integers `z` that is a multidimensional index into `m.extents()` and each element of `z` equals `0`.

3 Let *is-extents* be the exposition-only variable template:

```
template<class T> constexpr bool is-extents = false;

template<class SizeType, size_t ... args> constexpr bool is-extents<extents<SizeType, args...>> = true;
```

4 Let *layout-mapping-alike* be the exposition-only concept defined as follows:

```
template<class M>
concept layout-mapping-alike = requires {
  requires is-extents<typename M::extents_type>;
  { M::is_always_strided() } -> same_as<bool>;
  { M::is_always_contiguous() } -> same_as<bool>;
  { M::is_always_unique() } -> same_as<bool>;
  bool_constant<M::is_always_strided()>::value;
  bool_constant<M::is_always_contiguous()>::value;
  bool_constant<M::is_always_unique()>::value;
};
```

[Note: This concept checks that the functions `M::is_always_strided()`, `M::is_always_contiguous()`, and `M::is_always_unique()` exist, are constant expressions, and have a return type of `bool`. - *end note]*

24.7.𝔰.7.3 Constructors [mdspan.layoutstride.ctor]

```
template<class OtherSizeType>
constexpr mapping(const extents_type& e, span<OtherSizeType, rank_> s) noexcept;
template<class OtherSizeType>
constexpr mapping(const extents_type& e, const array<OtherSizeType, rank_>& s) noexcept;
```

1 *Constraints:*

(1.1) — `is_convertible_v<const OtherSizeType&, size_type>` is `true`,

(1.2) — `is_nothrow_constructible_v<size_type, const OtherSizeType&>` is `true`.

2 *Preconditions:*

(2.1) — `s[`i`] > 0` is `true` for all i in the range [0, *rank_*).

(2.2) — *REQUIRED-SPAN-SIZE*(`e, s`) is representable as a value of type `size_type` ([basic.fundamental]).

(2.3) — If *rank_* is greater than 0, then there exists a permutation P of the integers in the range [0, *rank_*), such that `s[` p_i `] >= s[` p_{i-1} `] * e.extent(` p_{i-1} `)` is `true` for all i in the range [1, *rank_*), where p_i is the i^{th} element of P. *[Note:* For `layout_stride`, this condition is necessary and sufficient for `is_unique()` to be `true`. — *end note]*

3 *Effects:* Direct-non-list-initializes *extents_* with `e`, and for all d in the range [0, *rank_*), direct-non-list-initializes *strides_*[d] with `as_const(s[`d`])`.

```
template<class StridedLayoutMapping>
  explicit(see below)
  constexpr mapping(const StridedLayoutMapping& other) noexcept;
```

4 *Constraints:*

(4.1) — *layout-mapping-alike*`<StridedLayoutMapping>` is satisfied.

(4.2) — `is_constructible_v<extents_type, typename StridedLayoutMapping::extents_type>` is `true`.

(4.3) — `StridedLayoutMapping::is_always_unique()` is `true`.

(4.4) — `StridedLayoutMapping::is_always_strided()` is `true`.

5 *Preconditions:*

(5.1) — `StridedLayoutMapping` meets the layout mapping requirements,

(5.2) — `other.stride(r) > 0` is `true` for all rank index `r` of `extents()`,

(5.3) — `other.required_span_size()` is representable as a value of type `size_type` ([basic.fundamental]), and

(5.4) — *`OFFSET`*`(other) == 0` is `true`.

6 *Effects:* Direct-non-list-initializes *`extents_`* with `other.extents()`, and for all d in the range [0, *`rank_`*), direct-non-list-initializes *`strides_`*`[`d`]` with `other.stride(`d`)`.

7 *Remarks:* The expression inside `explicit` is equivalent to:

```
!(is_convertible_v<typename StridedLayoutMapping::extents_type, extents_type> && (
  is-mapping-of<layout_left, LayoutStrideMapping> ||
  is-mapping-of<layout_right, LayoutStrideMapping> ||
  is-mapping-of<layout_stride, LayoutStrideMapping>))
```

24.7.🔲.7.4 Observers [mdspan.layoutstride.obs]

```
constexpr size_type required_span_size() const noexcept;
```

1 *Returns:* *`REQUIRED-SPAN-SIZE`*`(extents(),`*`strides_`*`)`.

```
template<class... Indices>
  constexpr size_type operator()(Indices... i) const noexcept;
```

2 *Constraints:*

(2.1) — `sizeof...(Indices) ==`*`rank_`* is `true`, and

(2.2) — `(is_convertible_v<Indices, size_type> && ...)` is `true`.

(2.3) — `(is_nothrow_constructible_v<size_type, Indices> && ...)` is `true`.

3 *Preconditions:* `extents_type::`*`index-cast`*`(i)` is a multidimensional index in *`extents_`* ([mdspan.terms]).

4 *Effects:* Let `P` be a parameter pack such that `is_same_v<index_sequence_for<Indices...>, index_sequence<P...>>` is `true`.
Equivalent to: `return ((static_cast<size_type>(i)*stride(P)) + ... + 0);`

```
constexpr bool is_contiguous() const noexcept;
```

5 *Returns:*

(5.1) — `true` if *`rank_`* is 0.

(5.2) — Otherwise, `true` if there is a permutation P of the integers in the range [0, *`rank_`*), such that `stride(` p_0 `)` equals `1`, and `stride(` p_i `)` equals `stride(` p_{i-1} `) * extents().extent(` p_{i-1} `)` for i in the range [1, *`rank_`*), where p_i is the i^{th} element of P.

(5.3) — Otherwise, `false`.

```
template<class OtherMapping>
  friend constexpr bool operator==(const mapping& x, const OtherMapping& y) noexcept;
```

6 *Constraints:*

(6.1) — *`layout-mapping-alike`*`<OtherMapping>` is satisfied.

(6.2) — *`rank_`* `== OtherMapping::extents_type::rank()` is `true`.

(6.3) — `OtherMapping::is_always_strided()` is `true`.

7 *Preconditions:* `OtherMapping` meets layout mapping requirements.

8 *Returns:* `true` if `x.extents() == y.extents()` is `true`, *`OFFSET`*`(y) == 0` is `true`, and each of `x.stride(`r`) == y.stride(`r`)` is `true` for r in the range of [0, `x.extents.rank()`). Otherwise, `false`.

24.7.🔲 Accessor Policy [mdspan.accessor]

24.7.﷿.1 General [mdspan.accessor.general]

1 An *accessor policy* defines types and operations by which a reference to a single object is created from an abstract data handle to a number of such objects and an index.

2 A range of indices $[0, N)$ is an *accessible range* of a given data handle and an accessor, if for each `i` in the range the accessor policy's `access` function produces a valid reference to an object.

3 In subclause 24.7.﷿.2,

(3.1) — `A` denotes an accessor policy.

(3.2) — `a` denotes a value of type `A` or `const A`.

(3.3) — `p` denotes a value of type `A::pointer` or `const A::pointer`. *[Note:* The type `A::pointer` need not be dereferenceable. - *end note]*

(3.4) — `n`, `i` and `j` each denote values of type `size_t`.

24.7.﷿.2 Requirements [mdspan.accessor.reqmts]

1 A type `A` meets the accessor policy requirements if

(1.1) — `A` models `copyable`,

(1.2) — `is_nothrow_move_constructible_v<A>` is `true`,

(1.3) — `is_nothrow_move_assignable_v<A>` is `true`,

(1.4) — `is_nothrow_swappable_v<A>` is `true`, and

(1.5) — the following types and expressions are well-formed and have the specified semantics.

```
typename A::element_type
```

2 *Result:* A complete object type that is not an abstract class type.

```
typename A::pointer
```

3 *Result:* A type that models `copyable`, and for which `is_nothrow_move_constructible_v<A::pointer>` is `true`, `is_nothrow_move_assignable_v<A::pointer>` is `true`, and `is_nothrow_swappable_v<A::pointer>` is `true`.

4 *[Note:* The type of `pointer` need not be `element_type*`. — *end note]*

```
typename A::reference
```

5 *Result:* A type that models `common_reference_with<A::reference&&, A::element_type&>`.

6 *[Note:* The type of `reference` need not be `element_type&`. — *end note]*

```
typename A::offset_policy
```

7 *Result:* A type `OP` such that:

(7.1) — `OP` meets the accessor policy requirements,

(7.2) — `constructible_from<OP, const A&>` is modeled, and

(7.3) — `is_same_v<typename OP::element_type, typename A::element_type>` is `true`.

```
a.access(p, i)
```

8 *Result:* `A::reference`

9 *Remarks:* The expression is equality preserving.

10 *[Note:* Concrete accessor policies can impose preconditions for their `access` function. However, they might not. For example, an accessor where `p` is `span<A::element_type,dynamic_extent>` and `access(p,i)` returns `p[i % p.size()]` does not need to impose a precondition on `i`. - *end note]*

```
a.offset(p, i)
```

11 *Result:* `A::offset_policy::pointer`

12 *Returns:* `q` such that for `b` being `A::offset_policy(a)`, and any integer `n` for which `[0, n)` is an accessible range of

`p` and `a`:

(12.1) — `[0, n-i)` is an accessible range of `q` and `b`; and

(12.2) — `b.access(q, j)` provides access to the same element as `a.access(p, i + j)`, for every `j` in the range `[0, n-i)`.

13 *Remarks:* The expression is equality preserving.

24.7.﷿.3 Class template `default_accessor` [mdspan.accessor.default]

24.7.﷿.3.1 Overview [mdspan.accessor.default.overview]

```
namespace std {
template<class ElementType>
  struct default_accessor {
    using offset_policy = default_accessor;
    using element_type = ElementType;
    using reference = ElementType&;
    using pointer = ElementType*;

    constexpr default_accessor() noexcept = default;

    template<class OtherElementType>
    constexpr default_accessor(default_accessor<OtherElementType>) noexcept {}

    constexpr reference access(pointer p, size_t i) const noexcept;

    constexpr pointer offset(pointer p, size_t i) const noexcept;
  };
}
```

1 `default_accessor` meets the accessor policy requirements.

2 `ElementType` is required to be a complete object type that is neither an abstract class type nor an array type.

3 Each specialization of `default_accessor` is a trivially copyable type that models `semiregular`.

4 `[0, n)` is an accessible range for an object `p` of type `pointer` and an object of type `default_accessor` if and only if `[p, p+n)` is a valid range.

24.7.﷿.3.2 Members [mdspan.accessor.default.members]

```
template<class OtherElementType>
constexpr default_accessor(default_accessor<OtherElementType>) noexcept {}
```

1 *Constraints:* `is_convertible_v<OtherElementType(*)[], element_type(*)[]>` is `true`.

```
constexpr reference access(pointer p, size_t i) const noexcept;
```

2 *Effects:* equivalent to `return p[i];`

```
constexpr pointer offset(pointer p, size_t i) const noexcept;
```

3 *Effects:* equivalent to `return p + i;`.

24.7.﷿ Class template `mdspan` [mdspan.mdspan]

24.7.﷿.1 Overview [mdspan.mdspan.overview]

1 `mdspan` is a view of a multidimensional array of elements.

```
namespace std {

template<class ElementType, class Extents, class LayoutPolicy, class AccessorPolicy>
class mdspan {
public:
  using extents_type = Extents;
  using layout_type = LayoutPolicy;
  using accessor_type = AccessorPolicy;
  using mapping_type = typename layout_type::template mapping<extents_type>;
  using element_type = ElementType;
  using value_type = remove_cv_t<element_type>;
  using size_type = typename extents_type::size_type;
  using rank_type = typename extents_type::rank_type;
  using pointer = typename accessor_type::pointer;
  using reference = typename accessor_type::reference;

  static constexpr rank_type rank() noexcept { return extents_type::rank(); }
```

```
  static constexpr rank_type rank_dynamic() noexcept { return extents_type::rank_dynamic(); }
  static constexpr size_t static_extent(rank_type r) noexcept { return extents_type::static_extent(r); }
  constexpr size_type extent(rank_type r) const noexcept { return extents().extent(r); }

  // [mdspan.mdspan.ctor], mdspan Constructors
  constexpr mdspan();
  constexpr mdspan(const mdspan& rhs) = default;
  constexpr mdspan(mdspan&& rhs) = default;

  template<class... OtherSizeTypes>
    explicit constexpr mdspan(pointer ptr, OtherSizeTypes... exts);
  template<class OtherSizeType, size_t N>
    explicit(N != rank_dynamic())
    constexpr mdspan(pointer p, span<OtherSizeType, N> exts);
  template<class OtherSizeType, size_t N>
    explicit(N != rank_dynamic())
    constexpr mdspan(pointer p, const array<OtherSizeType, N>& exts);
  constexpr mdspan(pointer p, const extents_type& ext);
  constexpr mdspan(pointer p, const mapping_type& m);
  constexpr mdspan(pointer p, const mapping_type& m, const accessor_type& a);

  template<class OtherElementType, class OtherExtents,
           class OtherLayoutPolicy, class OtherAccessorPolicy>
    explicit(see below)
    constexpr mdspan(
      const mdspan<OtherElementType, OtherExtents,
                   OtherLayoutPolicy, OtherAccessorPolicy>& other);

  constexpr mdspan& operator=(const mdspan& rhs) = default;
  constexpr mdspan& operator=(mdspan&& rhs) = default;

  // [mdspan.mdspan.members], mdspan members
  template<class... OtherSizeTypes>
    constexpr reference operator[](OtherSizeTypes... indices) const;
  template<class OtherSizeType>
    constexpr reference operator[](span<OtherSizeType, rank()> indices) const;
  template<class OtherSizeType>
    constexpr reference operator[](const array<OtherSizeType, rank()>& indices) const;

  constexpr size_t size() const noexcept;

  friend constexpr void swap(mdspan& x, mdspan& y) noexcept;

  constexpr const extents_type& extents() const noexcept { return map_.extents(); }
  constexpr const pointer& data() const noexcept { return ptr_; }
  constexpr const mapping_type& mapping() const noexcept { return map_; }
  constexpr const accessor_type& accessor() const noexcept { return acc_; }

  static constexpr bool is_always_unique() {
    return mapping_type::is_always_unique();
  }
  static constexpr bool is_always_contiguous() {
    return mapping_type::is_always_contiguous();
  }
  static constexpr bool is_always_strided() {
    return mapping_type::is_always_strided();
  }

  constexpr bool is_unique() const {
    return map_.is_unique();
  }
  constexpr bool is_contiguous() const {
    return map_.is_contiguous();
  }
  constexpr bool is_strided() const {
    return map_.is_strided();
  }
  constexpr size_type stride(rank_type r) const {
    return map_.stride(r);
  }

private:
  accessor_type acc_; // exposition only
  mapping_type map_; // exposition only
  pointer ptr_; // exposition only
};

template <class CArray>
requires(is_array_v<CArray> && rank_v<CArray> == 1)
mdspan(CArray&)
  -> mdspan<remove_all_extents_t<CArray>, extents<size_t, extent_v<CArray, 0>>>;

template <class Pointer>
```

```
requires(is_pointer_v<remove_reference_t<Pointer>>)
mdspan(Pointer&&)
  -> mdspan<remove_pointer_t<remove_reference_t<Pointer>>, extents<size_t>>;

template <class ElementType, class... Integrals>
requires((is_convertible_v<Integrals, size_t> && ...) && sizeof...(Integrals) > 0)
explicit mdspan(ElementType*, Integrals...)
  -> mdspan<ElementType, dextents<size_t, sizeof...(Integrals)>>;

template <class ElementType, class OtherSizeType, size_t N>
mdspan(ElementType*, span<OtherSizeType, N>)
  -> mdspan<ElementType, dextents<size_t, N>>;

template <class ElementType, class OtherSizeType, size_t N>
mdspan(ElementType*, const array<OtherSizeType, N>&)
  -> mdspan<ElementType, dextents<size_t, N>>;

template <class ElementType, class SizeType, size_t... ExtentsPack>
mdspan(ElementType*, const extents<SizeType, ExtentsPack...>&)
  -> mdspan<ElementType, extents<SizeType, ExtentsPack...>>;

template <class ElementType, class MappingType>
mdspan(ElementType*, const MappingType&)
  -> mdspan<ElementType, typename MappingType::extents_type,
            typename MappingType::layout_type>;

template <class MappingType, class AccessorType>
mdspan(const typename AccessorType::pointer&, const MappingType&, const AccessorType&)
  -> mdspan<typename AccessorType::element_type, typename MappingType::extents_type,
            typename MappingType::layout_type, AccessorType>;

}
```

2 *Mandates:*

(2.1) — `ElementType` is a a complete object type that is neither an abstract class type nor an array type,

(2.2) — `Extents` is a specialization of `extents`, and

(2.3) — `is_same_v<ElementType, typename AccessorPolicy::element_type>` is `true`.

3 `LayoutPolicy` shall meet the layout mapping policy requirements [mdspan.layoutpolicy.reqmts], and

4 `AccessorPolicy` shall meet the accessor policy requirements [mdspan.accessor.reqmts].

5 Each specialization `MDS` of `mdspan` models `copyable` and

(5.1) — `is_nothrow_move_constructible_v<MDS>` is `true`,

(5.2) — `is_nothrow_move_assignable_v<MDS>` is `true`, and

(5.3) — `is_nothrow_swappable_v<MDS>` is `true`.

6 A specialization of `mdspan` is a trivially copyable type if its `accessor_type`, `mapping_type`, and `pointer` are trivially copyable types.

24.7.﷿.2 Constructors [mdspan.mdspan.ctor]

```
constexpr mdspan();
```

1 *Constraints:*

(1.1) — `rank_dynamic() > 0` is `true`.

(1.2) — `is_default_constructible_v<pointer>` is `true`.

(1.3) — `is_default_constructible_v<mapping_type>` is `true`.

(1.4) — `is_default_constructible_v<accessor_type>` is `true`.

2 *Precondition:* [`0`, *map_*`.required_span_size()`) is an accessible range of *ptr_* and *acc_* for the values of *map_* and *acc_* after the invocation of this constructor.

3 *Effects:* Value-initializes *ptr_*, *map_*, and *acc_*.

```
template<class... OtherSizeTypes>
  explicit constexpr mdspan(pointer p, OtherSizeTypes... exts);
```

4 *Constraints:*

(4.1) — `(is_convertible_v<OtherSizeTypes, size_type> && ...)` is `true`,

(4.2) — `(is_nothrow_constructible<size_type, OtherSizeTypes> && ...)` is `true`,

(4.3) — `(sizeof...(OtherSizeTypes) == rank()) || (sizeof...(OtherSizeTypes) == rank_dynamic())` is `true`,

(4.4) — `is_constructible_v<mapping_type, extents_type>` is `true`, and

(4.5) — `is_default_constructible_v<accessor_type>` is `true`.

5 *Precondition:* [`0`, *`map_`*`.required_span_size()`) is an accessible range of `p` and *`acc_`* for the values of *`map_`* and *`acc_`* after the invocation of this constructor.

6 *Effects:*

(6.1) — Direct-non-list-initializes *`ptr_`* with `std::move(p)`,

(6.2) — Direct-non-list-initializes *`map_`* with `extents_type(static_cast<size_type>(std::move(exts))...)`, and

(6.3) — Value-initializes *`acc_`*.

```
template<class OtherSizeType, size_t N>
  explicit(N != rank_dynamic())
  constexpr mdspan(pointer p, span<OtherSizeType, N> exts);
template<class OtherSizeType, size_t N>
  explicit(N != rank_dynamic())
  constexpr mdspan(pointer p, const array<OtherSizeType, N>& exts);
```

7 *Constraints:*

(7.1) — `is_convertible_v<const OtherSizeType&, size_type>` is `true`,

(7.2) — `(is_nothrow_constructible<size_type, const OtherSizeType&> && ...)` is `true`,

(7.3) — `(N == rank()) || (N == rank_dynamic())` is `true`,

(7.4) — `is_constructible_v<mapping_type, extents_type>` is `true`, and

(7.5) — `is_default_constructible_v<accessor_type>` is `true`.

8 *Precondition:* [`0`, *`map_`*`.required_span_size()`) is an accessible range of `p` and *`acc_`* for the values of *`map_`* and *`acc_`* after the invocation of this constructor.

9 *Effects:*

(9.1) — Direct-non-list-initializes *`ptr_`* with `std::move(p)`,

(9.2) — Direct-non-list-initializes *`map_`* with `extents_type(exts)`, and

(9.3) — Value-initializes *`acc_`*.

```
constexpr mdspan(pointer p, const extents_type& ext);
```

10 *Constraints:*

(10.1) — `is_constructible_v<mapping_type, const extents_type&>` is `true`, and

(10.2) — `is_default_constructible_v<accessor_type>` is `true`.

11 *Precondition:* [`0`, *`map_`*`.required_span_size()`) is an accessible range of `p` and *`acc_`* for the values of *`map_`* and *`acc_`* after the invocation of this constructor.

12 *Effects:*

(12.1) — Direct-non-list-initializes *`ptr_`* with `std::move(p)`,

(12.2) — Direct-non-list-initializes *`map_`* with `ext`, and

(12.3) — Value-initializes *`acc_`*.

```
constexpr mdspan(pointer p, const mapping_type& m);
```

13 *Constraints:* `is_default_constructible_v<accessor_type>` is `true`.

14 *Precondition:* [`0`, `m.required_span_size()`) is an accessible range of `p` and *`acc_`* for value of *`acc_`* after the invocation of this constructor.

15 *Effects:*

(15.1) — Direct-non-list-initializes *ptr_* with `std::move(p)`, and

(15.2) — Direct-non-list-initializes *map_* with `m`.

(15.3) — Value-initializes *acc_*.

```
constexpr mdspan(pointer p, const mapping_type& m, const accessor_type& a);
```

16 *Precondition:* [`0`, `m.required_span_size()`) is an accessible range of `p` and `a`.

17 *Effects:*

(17.1) — Direct-non-list-initializes *ptr_* with `std::move(p)`,

(17.2) — Direct-non-list-initializes *map_* with `m`, and

(17.3) — Direct-non-list-initializes *acc_* with `a`.

```
template<class OtherElementType, class OtherExtents,
         class OtherLayoutPolicy, class OtherAccessor>
  explicit(see below)
  constexpr mdspan(const mdspan<OtherElementType, OtherExtents,
                                OtherLayoutPolicy, OtherAccessor>& other);
```

18 *Constraints:*

(18.1) — `is_constructible_v<mapping_type, const OtherLayoutPolicy::template mapping<OtherExtents>&>` is `true`, and

(18.2) — `is_constructible_v<accessor_type, const OtherAccessor&>` is `true`.

19 *Mandates:*

(19.1) — `is_constructible_v<pointer, const OtherAccessor::pointer&>` is `true`, and

(19.2) — `is_constructible_v<extents_type, OtherExtents>` is `true`.

20 *Preconditions:*

(20.1) — For each rank index `r` of `extents_type`, `static_extent(r) == dynamic_extent || static_extent(r) == other.extent(r)` is `true`.

(20.2) — [`0`, *map_*`.required_span_size()`) is an accessible range of *ptr_* and *acc_* for values of *ptr_*, *map_*, and *acc_* after the invocation of this constructor.

21 *Effects:*

(21.1) — Direct-non-list-initializes *ptr_* with `other.`*ptr_*,

(21.2) — Direct-non-list-initializes *map_* with `other.`*map_*, and

(21.3) — Direct-non-list-initializes *acc_* with `other.`*acc_*.

22 *Remarks:* The expression inside `explicit` is:

```
  !is_convertible_v<const OtherLayoutPolicy::template mapping<OtherExtents>&, mapping_type> ||
  !is_convertible_v<const OtherAccessor&, accessor_type>
```

24.7.﷿.3 Members [mdspan.mdspan.members]

```
template<class... OtherSizeTypes>
  constexpr reference operator[](OtherSizeTypes... indices) const;
```

1 *Constraints:*

(1.1) — `(is_convertible_v<OtherSizeTypes, size_type> && ...)` is `true`,

(1.2) — `(is_nothrow_constructible_v<size_type, OtherSizeTypes> && ...)` is `true`, and

(1.3) — `sizeof...(OtherSizeTypes) == rank()` is `true`.

2 Let `I` be `extents_type::`*index-cast*`(std::move(indices))`.

3 *Preconditions:* `I` is a multidimensional index in `extents()`. *[Note:* This implies that *map_*`(I...) <`*map_*`.required_span_size()` is `true`.— *end note]*;

4 *Effects:* Equivalent to: `return` *acc_*`.access(`*ptr_*`,`*map_*`(static_cast<size_type>(std::move(indices))...));`

```
template<class OtherSizeType>
  constexpr reference operator[](span<OtherSizeType, rank()> indices) const;
```

```
template<class OtherSizeType>
  constexpr reference operator[](const array<OtherSizeType, rank()>& indices) const;
```

5 *Constraints:*

(5.1) — `is_convertible_v<const OtherSizeType&, size_type>` is `true`, and

(5.2) — `is_nothrow_constructible_v<size_type, const OtherSizeType&>` is `true`.

6 *Effects:* Let `P` be a parameter pack such that `is_same_v<make_index_sequence<rank()>, index_sequence<P...>>` is `true`.
Equivalent to: `return operator[](as_const(indices[P])...);`

```
constexpr size_t size() const noexcept;
```

7 *Precondition:* The size of the multidimensional index space `extents()` is representable as a value of type `size_t` ([basic.fundamental]).

8 *Returns:* `extents().`*`fwd-prod-of-extents`*`(rank())`.

```
friend constexpr void swap(mdspan& x, mdspan& y) noexcept;
```

9 *Effects:* Equivalent to:

```
swap(x.ptr_, y.ptr_);
swap(x.map_, y.map_);
swap(x.acc_, y.acc_);
```

5 Implementation

There is an mdspan implementation available at https://github.com/kokkos/mdspan/.

6 Related Work

The original version of this paper, N4355, predates the "P" naming for papers.

Related papers:

- **P0122** : span: bounds-safe views for sequences of objects The `mdspan` codomain concept of *span* is well-aligned with this paper.
- **P0367** : Accessors: The P0367 Accessors proposal includes polymorphic mechanisms for accessing the memory an object or span of objects. The `AccessorPolicy` extension point in this proposal is intended to include such memory access properties.
- **P0331** : Motivation and Examples for Multidimensional Array
- **P0332** : Relaxed Incomplete Multidimensional Array Type Declaration
- **P0454** : Wording for a Minimal `mdspan` Included proposed modification of `span` to better align `span` with `mdspan`.
- **P0546** : Preparing `span` for the future Proposed modification of `span`
- **P0856** : Restrict access property for `mdspan` and `span`
- **P0860** : atomic access policy for `mdspan`
- **P0900** : An Ontology of Properties for `mdspan`
- **P2128** : Multidimensional subscript operator
- **P2299** : `mdspan` and CTAD